NAME:DHARSHIKA.P

REG NO:1924260191

COURSE CODE:CSA0822

GITHUB LINK: https://github.com/dharshipandiyan09-dotcom/CSA0822

TASK:1

Aim

To analyse a milk-payment problem by identifying the data the program must store, the values it must calculate, the rules it must follow, and the assumptions used.

Inputs and outputs

Inputs for each farmer

- **Farmer ID** — text, because it identifies the farmer and may contain letters or leading zeros.
- **Litres of milk** — a non-negative number; it may be fractional if the system records partial litres.
- **Fat percentage** — a decimal number from **2.0% to 7.0%**.
- **SNF percentage** — a decimal number. The assignment image does not specify its valid range, so the program below assumes **0% to 15%**.
- **Shift** — text, such as "Morning" or "Evening".

Values calculated

- **Rate per litre** — a decimal amount looked up from the payment-rate table using fat and SNF.
- **Payment amount** — litres × rate per litre.
- **Ten-day total** — the sum of the farmer's payment amounts over ten days.

Constants

The minimum and maximum fat percentages, the assumed SNF limits, the number of days in a payment period, and any rate-table values stay fixed while the program runs. They should be named constants so they can be changed in one place.

Rules and assumptions

- Litres cannot be negative.
- Fat must be between **2.0% and 7.0%**, inclusive.
- SNF is assumed to be between **0% and 15%**, inclusive; confirm this range with the payment scheme.
- Farmer ID and shift must not be blank.
- A rate must exist in the rate table for the entered fat and SNF values.
- The ten-day total includes only valid entries for that farmer.
- **Assumption:** Since the screenshot does not provide a rate formula or rate table, the example program uses a simple sample rate formula. Replace it with the official payment scheme before using real data.

Program

main.c

Run ▶

```c
#include <stdio.h>

int main(void) {
    const double MIN_FAT = 2.0;
    const double MAX_FAT = 7.0;
    const int DAYS = 10;

    char farmer_id[] = "F101";
    char shift[] = "Morning";
    double litres = 10.0;
    double fat = 4.0;
    double snf = 8.0;

    if (litres < 0) {
        printf("Error: litres cannot be negative.\n");
    } else if (fat < MIN_FAT || fat > MAX_FAT) {
        printf("Error: fat must be between 2.0 and 7.0.\n");
    } else {
        double rate = 20 + (fat * 2) + (snf * 0.5);
        double amount = litres * rate;
        double ten_day_total = amount * DAYS;

        printf("Farmer ID: %s\n", farmer_id);
        printf("Litres: %.1f\n", litres);
        printf("Fat percentage: %.1f\n", fat);
        printf("SNF percentage: %.1f\n", snf);
        printf("Shift: %s\n", shift);
        printf("Rate per litre: %.2f\n", rate);
        printf("Payment amount: %.2f\n", amount);
        printf("Ten-day total: %.2f\n", ten_day_total);
    }

    return 0;
```

Output

```
Farmer ID: F101
Litres: 10.0
Fat percentage: 4.0
SNF percentage: 8.0
Shift: Morning
Rate per litre: 32.00
Payment amount: 320.00
Ten-day total: 3200.00
```

TASK:2

Aim

To explore Python data types, division operators, and floating-point behavior, then calculate a payment total using Decimal for accurate money calculations.

Procedure

1. Open a Python interpreter or select Python in an online compiler.
2. Check the types of 10, 10.0, "10", True, and [10, 8.5] using type().
3. Try /, //, and % with 7 and 2, and test -7 // 2.

4. Compare 0.1 + 0.2 with 0.3.
5. Complete the total function using Decimal values created from strings.
6. Run the program and compare the float and Decimal totals.

PROGRAM



Explanation

- 10 is an integer, 10.0 is a floating-point number, and "10" is text.
- True is a Boolean value. [10, 8.5] is a list containing an integer and a float.
- / gives the division result, including decimals. // rounds down to the nearest whole number, which is why -7 // 2 gives -4. % gives the remainder.
- 0.1 + 0.2 == 0.3 is False because floats store some decimal values approximately.
- The float total has a small rounding difference. Decimal created from strings gives the exact total 33400.30.

- Creating a Decimal from a float, as in Decimal(float(amount)), carries the float's approximation into the Decimal. Use Decimal(amount) when the amount is a string.

  Alternatives and complexity

  For n payments, integer paise, float, and Decimal each process the payments in **O(n)** time and use **O(1)** extra space for the running total. Integer paise gives an exact total when amounts are in whole paise. Decimal gives an exact decimal total when created from strings. Float with round() can reduce displayed errors, but it does not guarantee an exact bank total for every set of payments. Decimal values generally use more memory than integer or float values.

Task 3 – Choose Variables and Name the Constants

Aim

To use clear variable names and named constants for a milk-payment calculation.

Procedure

1. Define the standard rate, fat and SNF step values, standard percentages, and allowed fat limits as constants.
2. Store the farmer's details and milk measurements in variables.
3. Calculate the fat and SNF steps, rate per litre, and payment amount.
4. Demonstrate dynamic typing and swapping values.

Program

main.c

Run ▶

```c
#include <math.h>

int main(void) {
    const double BASE_RATE = 30.00;
    const double FAT_STEP_VALUE = 0.40;
    const double SNF_STEP_VALUE = 0.30;
    const double STANDARD_FAT = 3.5;
    const double STANDARD_SNF = 8.5;
    const double STEP_SIZE = 0.1;

    char farmer_id[] = "F101";
    double litres = 10.0;
    double fat = 4.2;
    double snf = 8.7;
    char shift[] = "Morning";

    int fat_steps = (int)round((fat - STANDARD_FAT) / STEP_SIZE);
    int snf_steps = (int)round((snf - STANDARD_SNF) / STEP_SIZE);

    double rate = BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE;
    double amount = round(litres * rate * 100.0) / 100.0;

    printf("Farmer ID: %s\n", farmer_id);
    printf("Shift: %s\n", shift);
    printf("Fat steps: %d\n", fat_steps);
    printf("SNF steps: %d\n", snf_steps);
    printf("Rate per litre: %.2f\n", rate);
    printf("Payment amount: %.2f\n", amount);

    return 0;
}
```

Output

```
Farmer ID: F101
Shift: Morning
Fat steps: 7
SNF steps: 2
Rate per litre: 33.40
Payment amount: 334.00
```

Python uses uppercase names by convention for constants, but does not prevent reassignment. Variables use descriptive snake_case. Python is dynamically typed, so value can refer first to an integer and later to text. a, b = b, a swaps values in one statement.

Hard-coded rates require editing every place they appear. Named constants or a rate dictionary require changing each rate in one place. Runtime remains **O(1)** for this calculation; the rate representation changes maintainability, not the calculation's time complexity.

Task 4 – Write Correct Expressions

Aim

To calculate fat and SNF steps, rate per litre, and payment amount using correct Python expressions.

Procedure

1. Find the difference between the measured and standard fat and SNF values.
2. Divide each difference by the step size and round to the nearest whole step.
3. Apply the step values to the base rate.
4. Multiply the rate by litres to calculate the payment.
5. Run the expressions below to observe floating-point and rounding behavior.

Program

main.c

Run ▶

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    const double BASE_RATE = 30.00;
    const double FAT_STEP_VALUE = 0.40;
    const double SNF_STEP_VALUE = 0.30;
    const double STANDARD_FAT = 3.5;
    const double STANDARD_SNF = 8.5;
    const double STEP_SIZE = 0.1;

    double fat = 4.2;
    double snf = 8.7;
    double litres = 8.5;

    double raw_fat_steps = (fat - STANDARD_FAT) / STEP_SIZE;
    int fat_steps = (int)round(raw_fat_steps);
    int snf_steps = (int)round((snf - STANDARD_SNF) / STEP_SIZE);

    double rate = BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE;
    double amount = round(litres * rate * 100.0) / 100.0;

    double raw_steps_for_31 = (3.1 - STANDARD_FAT) / STEP_SIZE;

    printf("Raw fat steps: %.15f\n", raw_fat_steps);
    printf("Rounded fat steps: %d\n", fat_steps);
    printf("SNF steps: %d\n", snf_steps);
    printf("Rate per litre: %.2f\n", rate);
    printf("Amount: %.2f\n", amount);

    printf("0.1 + 0.2 = %.17f\n", 0.1 + 0.2);
```

Output

```
Raw fat steps: 7.000000000000002
Rounded fat steps: 7
SNF steps: 2
Rate per litre: 33.40
Amount: 283.90
0.1 + 0.2 = 0.30000000000000004
round(2.5) = 3
round(3.5) = 4
trunc(-2.7) = -2
floor(-2.7) = -3
-2 squared by precedence = -4
(-2) squared = 4
For fat 3.1: raw steps = -3.999999999999999
For fat 3.1: trunc = -3, floor = -4, round = -4
```

Binary floating-point cannot exactly represent some decimal fractions, so the computed step is a tiny amount above 7; round() produces the intended whole step. round() uses ties-to-even for exact halves, int() truncates toward zero, and math.floor() rounds downward.

Exponentiation binds more tightly than unary minus, so -2 ** 2 is interpreted as -(2 ** 2).

For fat 4.2, raw steps are approximately 7.000000000000001: int() and floor() both produce 7. For fat 3.1, raw steps are approximately -3.999999999999999: int() gives -3, floor() gives -4, and round() gives -4. These scalar operations use **O(1)** time and **O(1)** space.

Task 5 – Read Input and Convert Types

Aim

To read clerk entries, validate and convert the inputs, and display neatly formatted payment results.

Procedure

1. Read farmer ID and shift as text.
2. Read litres, fat, and SNF as text using input().
3. Convert numeric text to float inside a try block.
4. Display an error if conversion fails or the values are invalid.
5. Calculate and print the result with fixed-width f-string formatting.

Program

main.c

Run ▶

```c
#include <stdio.h>
#include <string.h>
#include <math.h>

int main(void) {
    const double BASE_RATE = 30.00;
    const double FAT_STEP_VALUE = 0.40;
    const double SNF_STEP_VALUE = 0.30;
    const double STANDARD_FAT = 3.5;
    const double STANDARD_SNF = 8.5;
    const double STEP_SIZE = 0.1;
    const double MIN_FAT = 2.0;
    const double MAX_FAT = 7.0;

    char farmer_id[50];
    char shift[30];
    double litres;
    double fat;
    double snf;

    printf("Farmer ID: ");
    if (scanf("%49s", farmer_id) != 1) {
        printf("Error: invalid farmer ID.\n");
        return 1;
    }

    printf("Shift: ");
    if (scanf("%29s", shift) != 1) {
        printf("Error: invalid shift.\n");
        return 1;
    }

    printf("Litres: ");
```

Output

```
Farmer ID: f101
Shift: morning
Litres: 8.2
Fat percentage: 4.2
SNF percentage: 8.7

Farmer ID:        f101
Shift:            morning
Rate per litre:   Rs    33.40
Payment amount:   Rs   273.88
```

input() always returns a string. float() converts valid numeric text; "eight" causes ValueError. The program accepts "8,5" by replacing the comma with a decimal point. " 8.5 " works because whitespace is stripped. Both validation with string methods and try/except take **O(1)** time per entry. try/except is direct and handles malformed values clearly.

Task 6 – Organise the Logic into Functions

Aim

To divide the milk-payment calculation into reusable functions with clear parameters, return values, and docstrings.

Procedure

1. Define the rate constants at module level.

2. Create rate_per_litre() to calculate and return the rate.
3. Create payout() to calculate and return the payment amount.
4. Create ten_day_total() to add payment amounts for ten entries.
5. Call the functions and print their returned values.

Program

```c
#include <stdio.h>
#include <math.h>

const double BASE_RATE = 30.00;
const double FAT_STEP_VALUE = 0.40;
const double SNF_STEP_VALUE = 0.30;
const double STANDARD_FAT = 3.5;
const double STANDARD_SNF = 8.5;
const double STEP_SIZE = 0.1;

double rate_per_litre(double fat, double snf) {
    int fat_steps = (int)round((fat - STANDARD_FAT) / STEP_SIZE);
    int snf_steps = (int)round((snf - STANDARD_SNF) / STEP_SIZE);

    return BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE;
}

double payout(double litres, double fat, double snf) {
    double rate = rate_per_litre(fat, snf);
    return round(litres * rate * 100.0) / 100.0;
}

double ten_day_total(
    const double litres[],
    const double fat[],
    const double snf[],
    int count
) {
    double total = 0.0;

    for (int i = 0; i < count; i++) {
```

```
Rate per litre: 33.40
One payment: 283.90
Total for entries: 917.50
```

A function’s return sends a result back to the caller; print() only displays it. Function parameters and local variables exist within the function. The constants are global names that the functions can read. The functions do not assign to the constants, so they do not change them.

Without functions, calculation code may be repeated for every entry. Lambdas can define short expressions but are less clear for multi-step logic. Function calls add a small

fixed overhead; processing n entries still takes **O(n)** time, and the extra running-total space is **O(1)**.

Task 7 – Build a Module

Aim

To put the milk rate card and calculation functions in a module, then import and use them from a separate Python script.

Procedure

1. Create a file named milkrate.py.
2. Add the rate constants and calculation functions to it.
3. Use the if __name__ == "__main__": block for the module's demonstration.
4. Create a second file named use_milkrate.py.
5. Import the module and call payout() from the second file.

File 1: milkrate.py

import milkrate uses names with the milkrate. prefix. from milkrate import payout imports only that function name. import milkrate as mr uses the shorter alias mr.

When imported, __name__ is "milkrate", so its demonstration block does not run. When executed directly, __name__ is "__main__", so the demonstration runs. Python searches the current folder and the locations listed in sys.path. Without the guard, the demonstration table would print during import. Importing once has a small one-time cost; Python caches loaded modules in sys.modules.

Task 8 – Plan the Algorithm Before Coding

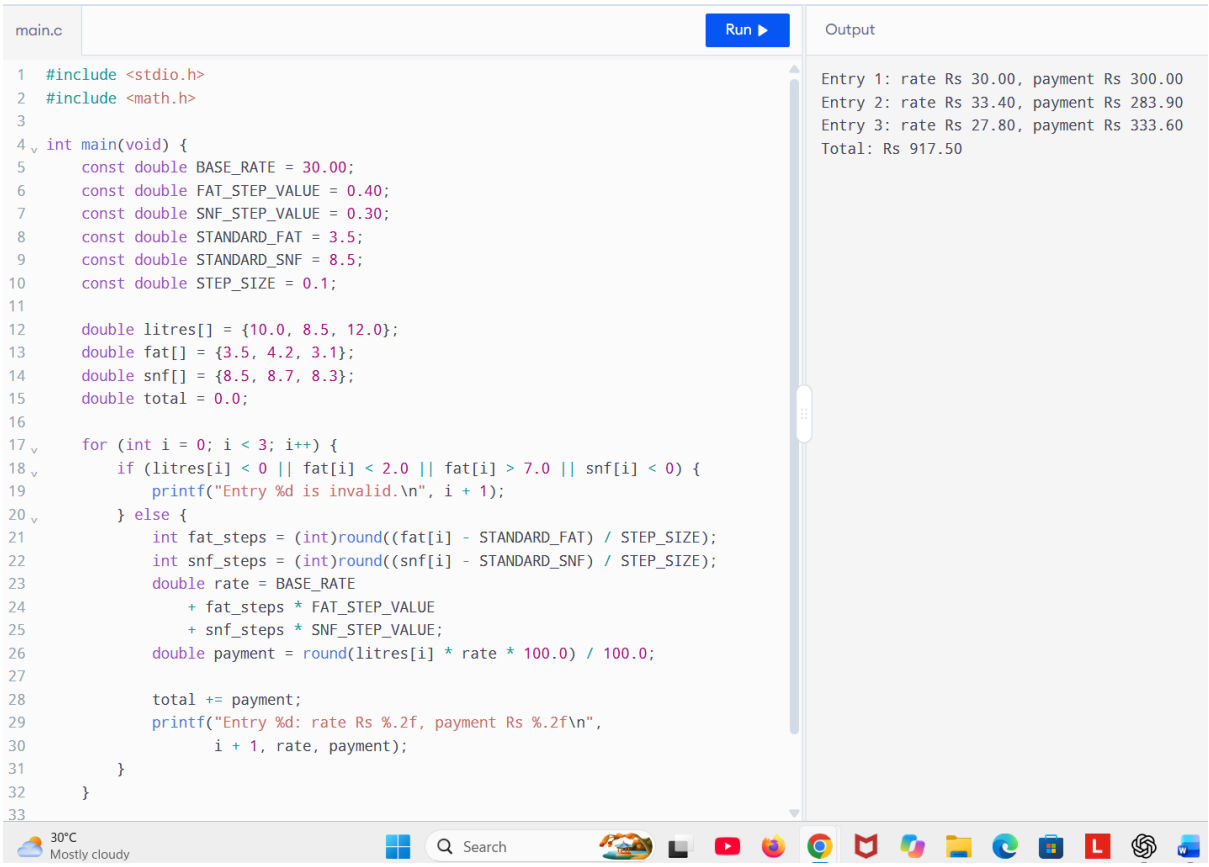
Aim

To plan and trace an algorithm that validates milk entries, calculates payments, and totals ten days of payments for one farmer.

Procedure

1. Understand the inputs, payment formula, and validity rules.
2. Write pseudocode before writing Python.
3. Trace the pseudocode using three sample entries.
4. Validate each entry and add each valid payment to a running total.
5. Review the flowchart and complexity.

Program



Storing every entry in a list takes **O(n)** time and **O(n)** space; it is useful when records must be reviewed, corrected, or reported later. Keeping only a running total also takes **O(n)** time but **O(1)** extra space; it is useful when only the total is needed.

Task 9 – Implement and Test the Calculator

Aim

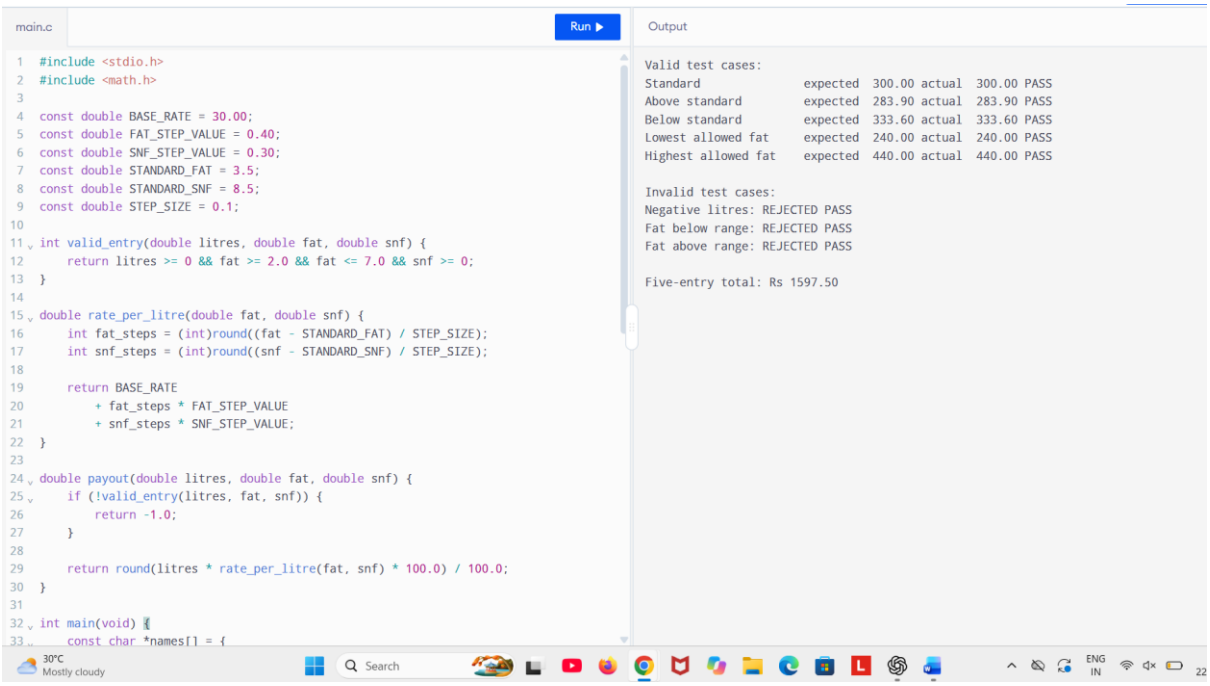
To implement a payment calculator and test normal, boundary, and invalid entries.

Procedure

1. Create functions for rate calculation and payout.
2. Validate litres, fat, and SNF.

3. Create test cases for fat and SNF above and below standard, boundary fat values, and invalid inputs.
4. Use assertions to check expected results.
5. Print the test results and a five-entry total.

Program



Assertion-based tests can be rerun with one command after code changes. Manual tests require typing each case again. Both approaches take time proportional to the number of tests, **O(n)**, but assertions make repeat runs faster and more consistent.

Task 10 – Compare Expectations with Execution

Aim

To compare float and Decimal totals and measure calculation time for different numbers of payments.

Procedure

1. Generate 300, 6,000, and 150,000 identical payment strings.
2. Sum them using floats and Decimal.
3. Measure each calculation with time.perf_counter().
4. Compare rounding each payment with rounding only the total.

Program

main.c

Run ▶

```c
#include <stdio.h>
#include <time.h>
#include <math.h>

void run_test(int count) {
    double float_total = 0.0;
    long long paise_total = 0;

    clock_t start_float = clock();
    for (int i = 0; i < count; i++) {
        float_total += 33.40;
    }
    clock_t end_float = clock();

    clock_t start_paise = clock();
    for (int i = 0; i < count; i++) {
        paise_total += 3340;
    }
    clock_t end_paise = clock();

    double float_seconds =
        (double)(end_float - start_float) / CLOCKS_PER_SEC;
    double paise_seconds =
        (double)(end_paise - start_paise) / CLOCKS_PER_SEC;

    printf("\nPayments: %d\n", count);
    printf("Float total: Rs %.2f\n", float_total);
    printf("Integer-paise total: Rs %lld.%02lld\n",
           paise_total / 100, paise_total % 100);
    printf("Float time: %.6f seconds\n", float_seconds);
    printf("Paise time: %.6f seconds\n", paise_seconds);
}
```

Output

```
Payments: 300
Float total: Rs 10020.00
Integer-paise total: Rs 10020.00
Float time: 0.000004 seconds
Paise time: 0.000001 seconds

Payments: 6000
Float total: Rs 200400.00
Integer-paise total: Rs 200400.00
Float time: 0.000023 seconds
Paise time: 0.000006 seconds

Payments: 150000
Float total: Rs 5010000.00
Integer-paise total: Rs 5010000.00
Float time: 0.000583 seconds
Paise time: 0.000122 seconds

Rounding each payment: 0.22
Rounding total: 0.21
```

The float totals can have tiny hidden errors even when formatting to two decimal places makes them appear equal. Decimal avoids binary representation error when constructed from strings. In this example, 0.105 is rounded to 0.11 for each payment, giving 0.22; adding first gives 0.210, which rounds to 0.21. The correct rule depends on the payment scheme: specify whether each transaction or only the final total is rounded.

Summing n payments takes **O(n)** time and uses **O(1)** extra space. A rate dictionary uses **O(1)** average lookup per entry but requires **O(k)** space for k distinct fat/SNF pairs. Recalculation is also constant time here, so caching may offer little benefit unless the rate calculation becomes more expensive or pairs repeat often.

Task 11 – Recommend and Defend the Final Design

Aim

To recommend a reliable design for the co-operative's milk payment system.

Procedure

1. Use named constants for rate values and limits.
2. Calculate rates through reusable functions.
3. Use decimal strings and Decimal for bank amounts.
4. Validate entries before calculating.
5. Test standard, boundary, invalid, and repeated-payment cases.
6. Keep rate values in a module or external configuration file.

Program

main.c

Run ▶

```c
#include <stdio.h>
#include <math.h>

const double BASE_RATE = 30.00;
const double FAT_STEP_VALUE = 0.40;
const double SNF_STEP_VALUE = 0.30;
const double STANDARD_FAT = 3.5;
const double STANDARD_SNF = 8.5;
const double STEP_SIZE = 0.1;

int valid_entry(double litres, double fat, double snf) {
    return litres >= 0 && fat >= 2.0 && fat <= 7.0 && snf >= 0;
}

double rate_per_litre(double fat, double snf) {
    int fat_steps = (int)round((fat - STANDARD_FAT) / STEP_SIZE);
    int snf_steps = (int)round((snf - STANDARD_SNF) / STEP_SIZE);

    return BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE;
}

long long payout_paise(double litres, double fat, double snf) {
    if (!valid_entry(litres, fat, snf)) {
        return -1;
    }

    double amount = litres * rate_per_litre(fat, snf);
    return (long long)round(amount * 100.0);
}

int main(void) {
```

Output

```
Payment: Rs 300.00
Payment: Rs 283.90
Payment: Rs 333.60
Total: Rs 917.50
```

Use Decimal for rupee values and define a clear rounding rule. Keep rates in one module or load them from a configuration file so a rate revision changes one source. The design's limitations include assumptions about the SNF range and the official rounding policy; these must be confirmed with the co-operative. For district scale, store records in a database, validate inputs, log rate-card versions, and use indexed lookups. The arithmetic itself remains **O(n)** for n entries.

If a rate card is stored in a text or JSON file, reading t rate values at startup takes **O(t)** time and **O(t)** space. This makes rate changes easier without editing program code, though the file must be validated and distributed safely.

Task 12 – Re-evaluate and Adapt the Design

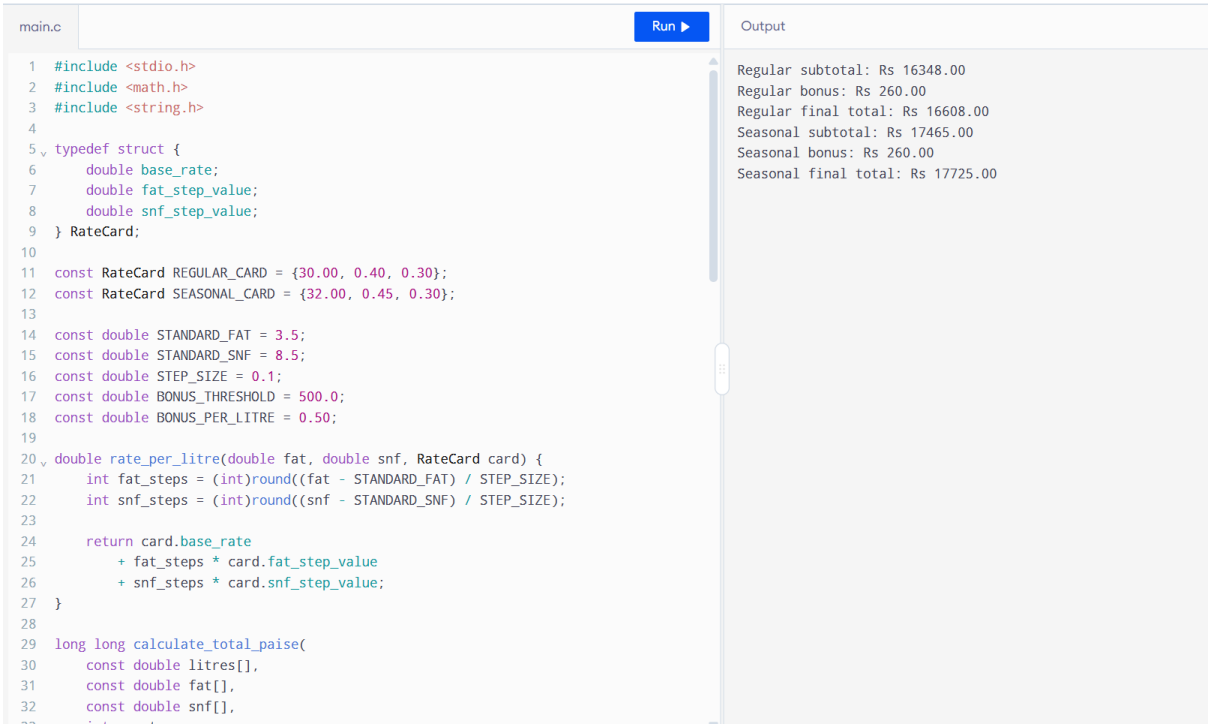
Aim

To adapt the milk-payment design for two-decimal analyser readings, seasonal rates, and a volume bonus.

Procedure

1. Store a separate rate card for each season.
2. Use Decimal values created from strings for measurements and money.
3. Calculate fat and SNF steps in increments of 0.1 percentage points, rounding to the nearest whole step.
4. Select the rate card using the season.
5. Add Rs 0.50 per litre when the ten-day volume is greater than 500 litres.
6. Test both seasons and the bonus threshold.

Program



The new analyser readings require rounding the step count consistently because readings can be hundredths of a percentage point. Seasonal base and fat-step values affect rate calculation, so the program selects a rate card by season. The bonus is based on total ten-day litres and applies only when the total is **more than** 500 litres.

The example gives 520 litres, so the bonus is 520 × Rs 0.50 = Rs 260. Decimal preserves the given hundredths and rupee amounts. The calculation takes **O(n)** time for n daily entries and uses **O(1)** additional space beyond the entry list. The co-operative should confirm the SNF seasonal step value and its official rounding policy before using this for actual payments.
